

CI/CD Workflow Report

Pipeline Architecture, Stages, Automation Results & Observations

Name	Karan Patel
Roll No.	2301410015
Course	B.Tech CSE (Cybersecurity)
Institution	K.R. Mangalam University, Gurugram
Assignment	Prepare a CI/CD Workflow Report (10 marks)

Pipeline: devops-sample-app1 | Orchestrator: Jenkins 2.568.3 | SCM: Git / GitHub

This report documents the CI/CD workflow implemented for the devops-sample-app1 project. It covers the overall pipeline architecture, the individual stages defined in the Jenkinsfile, the source-control integration that triggers the pipeline, the results of an automated run, and observations drawn from that run.

Contents

1. Pipeline Architecture Overview
2. Pipeline Stages (Jenkinsfile)
3. Source Control Configuration
4. Automation Results
5. Observations & Conclusion

1. Pipeline Architecture Overview

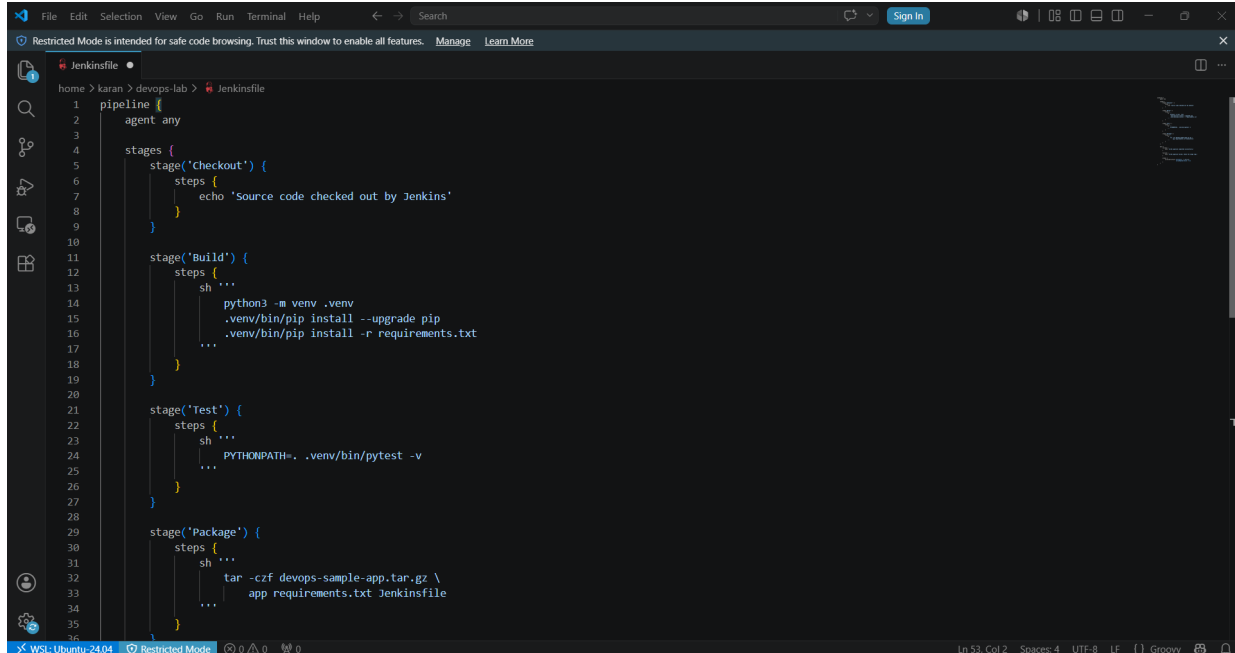
The pipeline follows a standard linear CI/CD architecture, triggered on changes pushed to the **main** branch of the GitHub repository. Jenkins, running as a local service, pulls the source code, builds an isolated Python environment, runs the automated test suite, and packages the verified application into a deployable artifact. Each stage runs sequentially inside a single pipeline job, and the job succeeds only if every stage completes without error.

At a high level, the flow is:

- **Source** — Developer pushes code to GitHub (karanpatel1704/devops-sample-app1).
- **Trigger** — Jenkins job 'devops-sample-app1' polls/builds against the **main** branch.
- **Checkout** — Jenkins clones the latest commit into its workspace.
- **Build** — A Python virtual environment is created and dependencies installed.
- **Test** — The pytest suite is executed against the build.
- **Package** — On success, the application is archived into a distributable tarball.
- **Post Actions** — Build artifacts are archived and a completion status is reported.

2. Pipeline Stages (Jenkinsfile)

The pipeline is defined declaratively in the project's Jenkinsfile, checked into the same Git repository as the application. This keeps the pipeline definition versioned alongside the code it builds. The Jenkinsfile defines four explicit stages plus a post-build action block:

A screenshot of the Visual Studio Code editor showing a Jenkinsfile. The file is opened in a window titled 'Jenkinsfile'. The code defines a pipeline with an 'agent any' block and four stages: 'Checkout', 'Build', 'Test', and 'Package'. Each stage contains shell steps. The 'Checkout' stage echoes the source code retrieval. The 'Build' stage sets up a virtual environment and installs dependencies. The 'Test' stage runs pytest. The 'Package' stage creates a tarball of the application and requirements. The status bar at the bottom indicates the file is in 'Restricted Mode' and the editor is on line 53, column 2.

```
1 pipeline {
2   agent any
3
4   stages {
5     stage('Checkout') {
6       steps {
7         echo 'Source code checked out by Jenkins'
8       }
9     }
10
11    stage('Build') {
12      steps {
13        sh '''
14          python3 -m venv .venv
15          .venv/bin/pip install --upgrade pip
16          .venv/bin/pip install -r requirements.txt
17        '''
18      }
19    }
20
21    stage('Test') {
22      steps {
23        sh '''
24          PYTHONPATH=. .venv/bin/pytest -v
25        '''
26      }
27    }
28
29    stage('Package') {
30      steps {
31        sh '''
32          tar -czf devops-sample-app.tar.gz \
33            app requirements.txt Jenkinsfile
34        '''
35      }
36    }
37  }
38 }
```

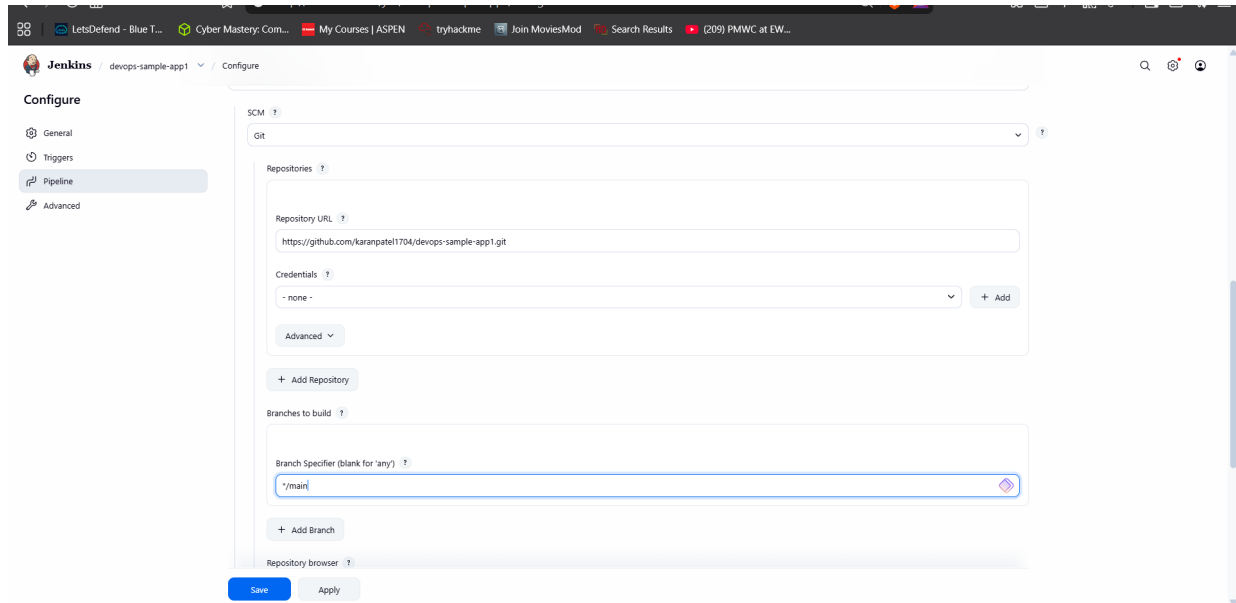
Jenkinsfile opened in VS Code showing the declarative pipeline: Checkout, Build, Test, and Package stages, each running shell steps inside the 'agent any' block.

Stage	Purpose	Key command(s)
Checkout	Confirms source retrieval	echo 'Source code checked out by Jenkins'
Build	Prepares an isolated runtime	python3 -m venv .venv pip install -r requirements.txt
Test	Validates functionality	PYTHONPATH=. .venv/bin/pytest -v
Package	Produces a deployable artifact	tar -czf devops-sample-app.tar.gz ...
Post Actions	Archives artifacts, reports status	archiveArtifacts + completion echo

3. Source Control Configuration

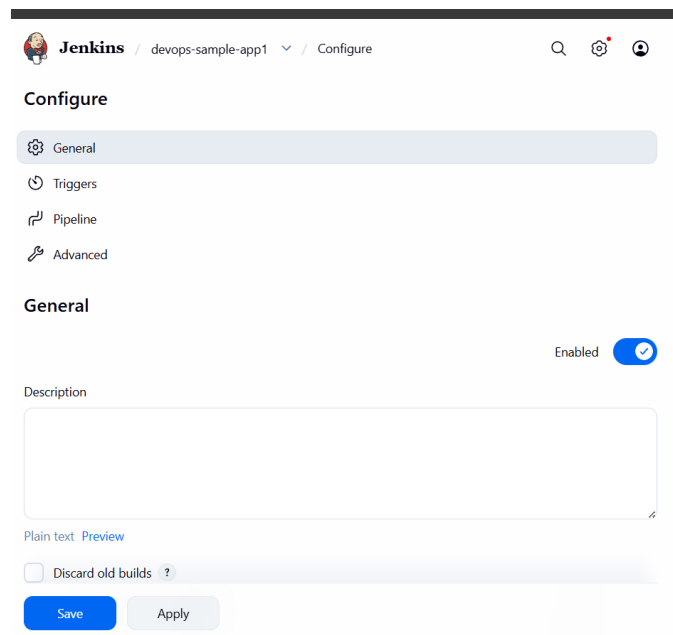
The Jenkins job is configured to pull directly from the project's GitHub repository, restricting builds to the **main** branch. This ensures only reviewed/merged code reaches the pipeline, and keeps the CI process tied to a single source of truth.

3.1 SCM & Branch Configuration



Pipeline configuration: SCM set to Git, repository URL `https://github.com/karanpatel1704/devops-sample-app1.git`, branch specifier `*/main`.

3.2 Job General Settings

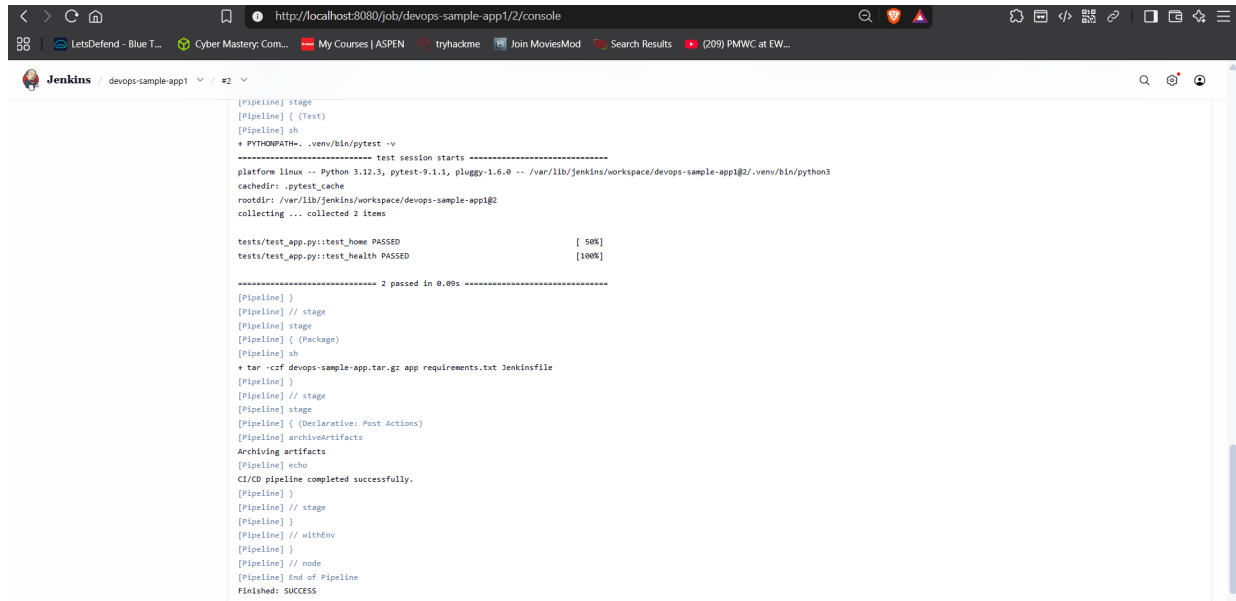


General job configuration for 'devops-sample-app1' — job enabled, with Pipeline, Triggers, and Advanced sections available for further automation (e.g., build triggers, timeouts).

4. Automation Results

Build #2 of the 'devops-sample-app1' job was executed end-to-end. The console output below shows the Test and Package stages, the post-build actions, and the final pipeline status.

4.1 Console Output — Build #2



```
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] sh
+ PYTHONPATH=. .venv/bin/pytest -v

===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 -- /var/lib/jenkins/workspace/devops-sample-app1@2/.venv/bin/python
cachedir: .pytest_cache
rootdir: /var/lib/jenkins/workspace/devops-sample-app1@2
collecting ... collected 2 items

tests/test_app.py::test_home PASSED [ 50%]
tests/test_app.py::test_health PASSED [100%]

===== 2 passed in 0.09s =====
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Package)
[Pipeline] sh
+ tar -czf devops-sample-app.tar.gz app requirements.txt Jenkinsfile
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] archiveArtifacts
Archiving artifacts
[Pipeline] echo
CI/CD pipeline completed successfully.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] end of Pipeline
Finished: SUCCESS
```

Console output for build #2: pytest run inside the Jenkins workspace (2 passed in 0.09s), packaging step (tar -czf devops-sample-app.tar.gz), artifact archiving, and 'Finished: SUCCESS'.

Key results extracted from the run:

- Both test cases (**test_home**, **test_health**) passed in 0.09 seconds.
- The Test stage ran inside an isolated workspace: `/var/lib/jenkins/workspace/devops-sample-app1@2`.
- The Package stage archived `app/`, `requirements.txt`, and `Jenkinsfile` into `devops-sample-app.tar.gz`.
- Post-build actions archived the artifact and logged 'CI/CD pipeline completed successfully.'
- The overall pipeline result was **SUCCESS**, with no failed stages.

5. Observations & Conclusion

Observations:

- Keeping the Jenkinsfile in version control (rather than configuring stages only through the Jenkins UI) makes the pipeline reproducible and auditable alongside application code changes.
- Restricting the branch specifier to `*/main` prevents builds from firing on unrelated or in-progress feature branches, keeping CI feedback focused on release-ready code.
- Test isolation via `PYTHONPATH` and a dedicated virtual environment avoided dependency conflicts with the Jenkins host environment, and the full Test stage completed in under a tenth of a second.
- Archiving the packaged tarball as a Jenkins artifact gives a traceable, versioned build output that can be picked up by a separate deployment step without re-running Build/Test.
- The current configuration uses `'- none -'` for Git credentials, which works for a public repository but would need a credentials binding for a private repo or push-back operations.

Conclusion:

The implemented pipeline demonstrates a complete, working CI/CD loop — from source checkout through automated testing to artifact packaging — orchestrated entirely by Jenkins and driven by a version-controlled Jenkinsfile. Build #2 confirms the pipeline is reliable and repeatable, finishing with a SUCCESS status and a ready-to-deploy artifact. This provides a solid foundation for extending the pipeline with an automated deployment stage and additional triggers (e.g., GitHub webhooks) in future iterations.

End of report.